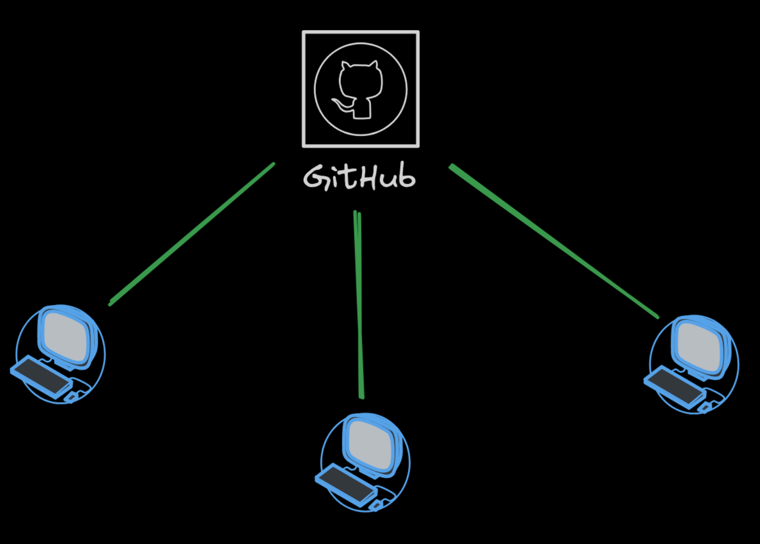
Building our own Git Server

To understand a Git server, you have to understand the problem it solves.
If you're working solo, Git runs locally on your machine, managing your file versions.
But what if you have a team? You can't just copy-paste folders on a USB drive.

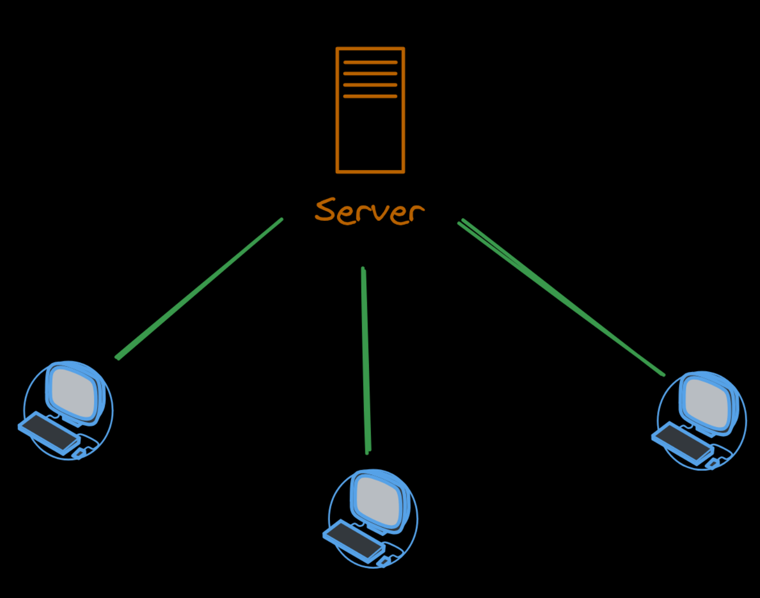
You need a centralized location. A remote server.
Usually, we just hand this over to GitHub or GitLab.
But today, we are taking control.



THE PROTOCOL (How Git communicates)

To push code across the internet, we need a secure network protocol.
Git natively supports HTTP and SSH. We are going to use SSH because it's the industry standard for infrastructure.

SSH stands for Secure Shell. Normally, you use it to get a remote terminal on a server.
Before SSH, servers used Telnet, which sent text in plain English—meaning anyone on your network could easily steal your passwords. SSH fixes this by creating an encrypted tunnel.
Everything sent through it looks like cryptographic gibberish to anyone trying to spy on it.

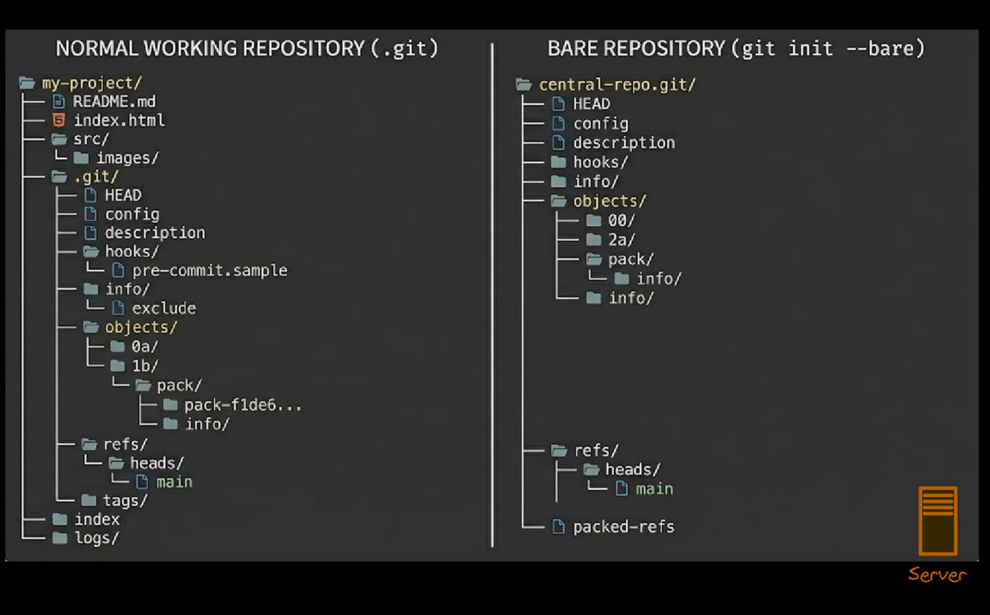


AUTHENTICATION

But how does the server know it's you connecting? Instead of passwords, we use asymmetric cryptography.
You run a simple command to generate an ED25519 keypair.
This gives you two files: a Private Key (which never leaves your laptop) and a Public Key (which acts like a padlock).

You take that Public Key and paste it into an authorized_keys file on the server.
When you try to connect, the server challenges you.
If your private key matches the public padlock, you're in.

THE GIT MAGIC (Bare Repositories)



Now, how does Git actually fit inside this server? On your local machine,
a Git repo has your working files plus a hidden .git folder tracking the history.

On a server, we don't need the working files because nobody is opening VS Code on the server.
We only need the history. This is called a Bare Repository.
It is strictly a database for Git objects.

The Local Sandbox

Open a terminal on your system and create a dedicated directory to act as your server's environment.
Inside, we'll initialize the bare Git repository.

```
# Create the working directory for our server
mkdir -p ~/git-server-dev/repos/test-repo.git
cd ~/git-server-dev

# Initialize the bare repository (no working tree, just Git objects)
cd repos/test-repo.git
git init --bare
cd ../..
```

Generate the Server's Identity

An SSH server needs its own private key to identify itself to clients and establish a secure connection.
We will generate an ED25519 key (the modern, highly secure standard) specifically for this project so you don't mess with your system's default keys.

In your ~/git-server-dev directory, run:

```
ssh-keygen -t ed25519 -f server_ed25519 -q -N ""
```

This creates two files: server_ed25519 (the private key) and server_ed25519.pub (the public key).

Scaffold the Go SSH Server

Now, let's initialize the Go project and install the official Go cryptography package,
which contains the low-level SSH implementation.

```
# Initialize the Go module
go mod init gitserver

# Download the SSH package
go get golang.org/x/crypto/ssh
```

Generating Your SSH Public Key

By default, a user's SSH keys are stored in that user's ~/.ssh directory.
You can easily check to see if you have a key already by going to that directory and listing the contents:

```
$ cd ~/.ssh
$ ls
authorized_keys2  id_dsa       known_hosts
config            id_dsa.pub
```

You're looking for a pair of files named something like id_dsa or id_rsa and a matching file with a .pub extension.
The .pub file is your public key, and the other file is the corresponding private key.
If you don't have these files (or you don't even have a .ssh directory),
you can create them by running a program called ssh-keygen, which is provided with the SSH package on Linux/macOS systems and comes with Git for Windows:

```
$ ssh-keygen -o
Generating public/private rsa key pair.
Enter file in which to save the key (/home/schacon/.ssh/id_rsa):
Created directory '/home/schacon/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/schacon/.ssh/id_rsa.
Your public key has been saved in /home/schacon/.ssh/id_rsa.pub.
The key fingerprint is:
d0:82:24:8e:d7:f1:bb:9b:33:53:96:93:49:da:9b:e3 schacon@mylaptop.local
```

First it confirms where you want to save the key (.ssh/id_rsa), and then it asks twice for a passphrase,
which you can leave empty if you don't want to type a password when you use the key.
However, if you do use a password, make sure to add the -o option; it saves the private key in a format that is more resistant to brute-force password cracking than is the default format. You can also use the ssh-agent tool to prevent having to enter the password each time.

Now, each user that does this has to send their public key to you or whoever is administrating the Git server (assuming you're using an SSH server setup that requires public keys).
All they have to do is copy the contents of the .pub file and email it.
The public keys look something like this:

```
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAklOUpkDHrfHY17SbrmTIpNLTGK9Tjom/BWDSU
GPl+nafzlHDTYW7hdI4yZ5ew18JH4JW9jbhUFrviQzM7xlELEVf4h9lFX5QVkbPppSwg0cda3
Pbv7kOdJ/MTyBlWXFCR+HAo3FXRitBqxiX1nKhXpHAZsMciLq8V6RjsNAQwdsdMFvSlVK/7XA
t3FaoJoAsncM1Q9x5+3V0Ww68/eIFmb1zuUFljQJKprrX88XypNDvjYNby6vw/Pb0rwert/En
mZ+AW4OZPnTPI89ZPmVMLuayrD2cE86Z/il8b+gw3r3+1nKatmIkjn2so1d01QraTlMqVSsbx
NrRFi9wrf+M7Q== schacon@mylaptop.local
```